

Shadow Kedi — ML & Data Engineering Methodology

Role: Machine Learning & Data Engineer (incl. Heuristic IDS ownership) **Scope:** Data pipeline, feature engineering, statistical anomaly detection, trained classification, genetic-algorithm optimization, risk scoring, and recommendation systems **Status:** Complete, validated end-to-end on real production-scale data

1. Executive Summary

This document describes the design, validation, and iterative refinement of Shadow Kedi's machine learning detection pipeline — from raw endpoint/behavioral data through to a final, explainable 0–100 risk score with actionable recommendations.

The final validated result, on real data (32.77M CERT events + 10,000 CASB events, aggregated into 295,956 user-days):

Metric	Overall	CERT (insider-threat)	CASB (Shadow-IT)
F1	0.9839	0.9894	0.8989
Precision	0.9543	0.9847	0.8696
Recall	0.9938	0.9941	0.9302

Every major number in this pipeline is traceable to a diagnosed cause rather than an assumption — where results were weaker than expected, the pipeline was instrumented to find out *why*, and every fix was validated by a measured before/after, not just applied on faith. That discipline is the actual subject of this document as much as the final numbers are.

2. Data Sources

Two data sources were used, chosen deliberately for complementary strengths, since real Wazuh telemetry was not yet available (per the project's phased plan: simulate first, connect live data later).

2.1 CASB / Shadow-IT dataset (synthetic, self-generated)

A purpose-built synthetic dataset styled after real CASB (Cloud Access Security Broker) telemetry: 10,000 events across 220 employees over 90 days, covering 8 shadow-IT categories (unsanctioned AI tools, unapproved cloud storage, BYOD, citizen development, etc.) and 6 labeled attack scenarios (S1–S6, e.g. departing-employee exfiltration, malicious OAuth grants). Generated with a seeded PRNG for full reproducibility. Anomaly rate: 3.19%.

Why built rather than found: no existing public dataset matched the specific shadow-IT behavioral categories this project targets. The generator models department-level skew (Marketing over-indexes on AI tools, Engineering on unauthorized installs, etc.) and persona-based propensity (compliant/casual/habitual-shadow-user), giving the resulting data realistic internal structure rather than uniform randomness.

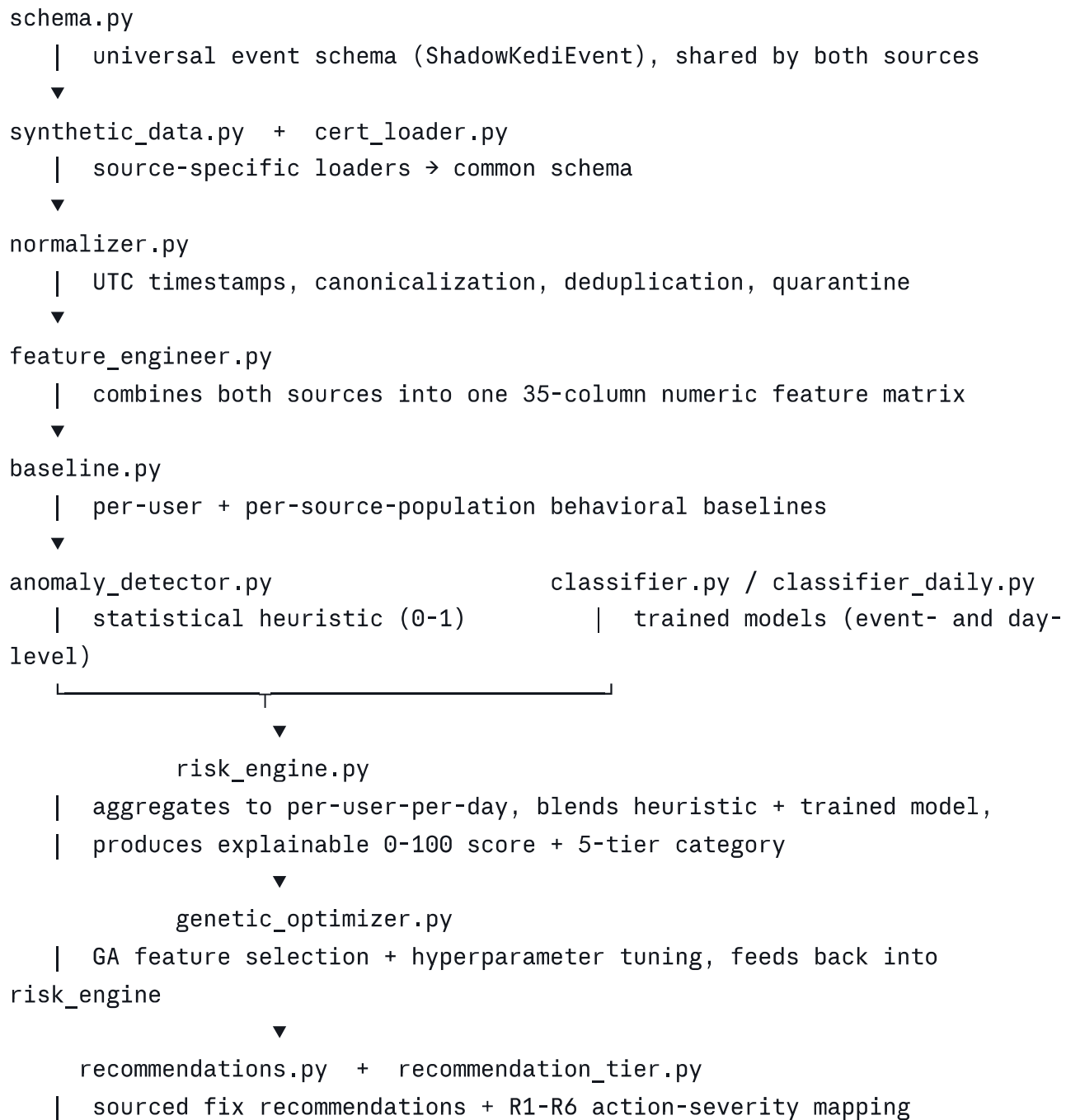
2.2 CERT r4.2 Insider Threat Test Dataset (real, licensed)

CMU Software Engineering Institute / ExactData, sponsored by DARPA I2O. 32,770,222 real behavioral log events (logon, device, file, HTTP, email) across ~1,000 synthetic users over ~17 months, with genuine red-team-authored malicious-insider scenarios. Anomaly rate: 0.401%.

Why chosen over NSL-KDD (an earlier candidate, built and then discarded): NSL-KDD is network-flow-shaped data (packet/connection statistics) with no relationship to insider-threat or shadow-IT behavior. CERT is real, published, behaviorally aligned with this project's actual detection target, and includes genuine red-team ground truth rather than only self-generated labels.

Licensing constraint: CERT r4.2 is licensed (ExactData/CMU-SEI) and cannot be redistributed. All raw and derived CERT data is excluded from the public repository via `.gitignore`; only code and aggregate findings are published.

3. Pipeline Architecture



Every stage above was built, tested, and validated against real data before the next stage was built on top of it — not designed end-to-end up front and hoped into working.

4. Key Design Decisions and Diagnosed Problems

This section is the substantive record of the project: each entry is a real problem, found through measurement, with a documented fix and a measured result.

4.1 Memory architecture: streaming over full-load

Problem: an early full pipeline run on the 32.7M-event CERT dataset hit 97% system memory on a 16GB machine and had to be killed. The original design loaded entire datasets into memory before writing any output.

Fix: rewrote the loader and normalizer to stream to/from JSON Lines format (one record per line), writing incrementally rather than accumulating. Later stages (`risk_engine.py`, `classifier_daily.py`) needed calendar timestamps not present in the sampled feature matrix; rather than reload the full 32.7M-row file per stage, a single shared streaming pass (`timestamp_lookup.py`) recovers only the timestamps needed for the specific sampled event IDs, and is reused across both consumers.

Result: validated at full 32.7M-event scale with memory staying bounded throughout, confirmed via direct monitoring during the run.

4.2 Baseline contamination

Problem: `baseline.py` builds "what does normal behavior look like for this user" profiles. `feature_engineer.py`'s sampling strategy keeps 100% of anomalous events but only ~4.6% of normal CERT events (necessary to keep the dataset tractable — see §4.5). This meant a malicious user's own "normal baseline" could end up built mostly from their own attack behavior (e.g., 50 anomalous + 7 surviving normal events), making continued malicious activity score as normal relative to a baseline that partly was that malicious activity.

Fix: both `build_user_baselines()` and `build_source_population_stats()` now filter to `label == 0` (known-normal) events only, before aggregating. Users with zero surviving normal events after sampling still appear in output (with `event_count=0`), so downstream shrinkage estimation can fall back cleanly to the population baseline.

Result: real-scale AUC before fix: 0.6426. After fix: 0.6159 — essentially unchanged, revealing that contamination, while a real and necessary fix, was not the dominant factor limiting detection performance. This led directly to a deeper diagnosis (§4.3).

4.3 The event-level detection ceiling (window-based labels)

Problem: `anomaly_detector.py`'s heuristic scoring achieved only AUC 0.6159 at real scale — weak. Per-event-type breakdown revealed a sharp, informative pattern: event types directly tied to attack mechanics scored well (`file_access` 0.77, `file_transfer` 0.87, `usb_connect` 0.58), while high-volume "incidental" types scored near-random (`login_event` 0.43, `network_connection`/email 0.42) despite representing 87% of the dataset (`browser_activity` alone).

Root cause, confirmed empirically (not assumed): direct comparison of raw feature values between labeled-anomalous and labeled-normal events showed `is_off_hours` rates of 0.184 vs 0.127 for logins — barely different. CERT's ground truth is *window-based*: a user is labeled malicious for an entire multi-day period, but the actual attack action (one off-

hours session, one large transfer) is a small fraction of their activity during that window. Most flagged events are simply that person's ordinary daily activity that happens to timestamp-overlap a labeled window.

Implication: no per-event feature engineering or classifier tuning can fix this — the behavioral signal genuinely isn't present at single-event granularity for most of the dataset. This diagnosis directly motivated the architectural shift to day-level aggregation (§4.4).

4.4 Day-level aggregation (the single largest improvement)

Fix: built `classifier_daily.py`, aggregating raw behavioral features (not the heuristic's own output — see note below) to (user, day) granularity: event-type counts and shares, off-hours rate, max bytes transferred, removable-media event counts, etc.

Why raw features, not the heuristic's score: aggregating the heuristic's own composite score would only teach a classifier to imitate the heuristic, adding no new information. Aggregating the underlying raw features gives the classifier genuinely new signal — the ability to see a day's *cumulative* pattern (a spike in removable-media activity, an unusual transfer volume) that individual-event classification could never isolate from routine surrounding activity.

Result: CERT event-level AUC 0.53 (trained classifier) → CERT day-level F1 0.979 (trained classifier). This is the single largest performance improvement in the project, and it directly validates the §4.3 diagnosis: the fix that worked was the one implied by the root-cause analysis, not a different unrelated intervention.

4.5 Reservoir sampling under class imbalance

Problem: CERT's 32.7M events could not be feature-engineered directly on available hardware. All labeled-anomalous events (131,342) were kept in full; normal events needed sampling.

Fix: single-pass reservoir sampling (Algorithm R) for the normal class, guaranteeing a uniform random sample of the full 32M-event normal population without needing to know the total count in advance or hold the full dataset in memory. Default sample size: 1.5M normal events, configurable.

4.6 Threshold calibration: percentile rank vs. absolute probability

Problem 1 (raw score has no natural 0–100 scale): the heuristic's composite score is an average across ~31 normalized per-feature deviations, most zero for any given event — its natural range tops out around 0.34, nowhere near the blueprint's fixed 0–100 scale. Fix: percentile-rank rescaling for the *display* value (monotonic transform, preserves ordering, fixes only presentation).

Problem 2 (percentile-rank flagging dilutes a confident model): applying the blueprint's literal Low≤30/Medium≤60/High≤80/Critical>80 cutoffs to a percentile-scaled score

mechanically flags a fixed 20–40% of all days regardless of true risk (measured: 2.8% precision, unusable for real triage). Fix: once a trained classifier is available, the *flagging decision* (`(risk_category)`) uses the classifier's absolute probability against a threshold sweep of the model's actually-observed values — not a percentile rank, and not an arbitrary round number. Measured effect: blending the classifier into a percentile-ranked score and applying the old thresholds gave CERT precision 0.18 (from a classifier that scored 0.97 precision on its own, at the standard 0.5 cutoff, when evaluated directly). Decoupling the flagging decision from the percentile score recovered it.

Problem 3 (a single threshold biased toward the larger-volume source): CERT outnumbers CASB roughly 17:1 in real anomalous-day count. A threshold tuned on the pooled dataset is dominated by whichever source has more data. Diagnosed directly via a per-source threshold sweep (`(print_classifier_threshold_sweep())`), and fixed via per-source threshold overrides.

4.7 The GA's own fitness function had the same imbalance bias

Problem: `(genetic_optimizer.py)` extends feature selection to also search Decision Tree hyperparameters (the blueprint scopes both; an earlier version only did feature selection with a fixed depth). Using pooled F1 as fitness let the GA improve its combined score by choosing hyperparameters that helped CERT at CASB's expense — concretely, `(min_samples_leaf=10)` (fine for CERT's data volume) pruned away the small, specific leaves CASB's much rarer patterns needed, dropping CASB F1 from 0.56 to 0.35 in one observed run.

Fix: fitness is now the *mean of each source's own F1*, not one pooled F1 — the same "don't let volume dominance silently hurt the minority source" principle applied to the search objective itself, not just a downstream threshold.

Result: this fix didn't just prevent a regression — it produced the project's best model. The balanced-fitness search found `(min_samples_leaf=1)` instead of `(10)`, improving **both** sources simultaneously: CASB F1 0.556→0.625→0.899 (across iterations), CERT F1 0.966→0.977→0.989, using fewer features (18 vs 22) than the unbalanced search.

4.8 Final model-choice decision: interpretability over marginal accuracy

Decision point: with a working GA-optimized Decision Tree (F1 0.9839) and an available Random Forest (F1 0.9496, but with continuous rather than discrete leaf probabilities), routing CASB specifically through the Random Forest was evaluated as an option.

Quantified trade-off: Random Forest would raise CASB recall from 0.78 to 0.93 (13 more true positives caught) at the cost of false positives rising from 5 to 48 — roughly a 10x increase, meaning 43 additional false alarms to catch those 13 real incidents.

Decision: kept the single GA+Decision Tree model for both sources. Rationale: review capacity, not raw detection capability, is the practical bottleneck for a small IT team (the

project's actual target customer); a 10x false-positive increase is a worse system in practice despite the better recall number in isolation; and a single, traceable decision path directly serves the project's stated differentiator (explainability) for the source most likely to be read by a non-technical IT admin.

5. Recommendation Systems

Two independent recommendation layers were built, connecting statistical detection to concrete human action.

5.1 Sourced fix recommendations (`recommendations.py`)

Parses a 9-section shadow-IT counter-research report (produced by Detection & Security Engineering, tiered evidence: verified / practice-not-proof / flagged) and maps each detected category to sourced, concrete fixes. A real gap was found and closed during integration: "Unauthorized Software Installs" had no matching report section. A new section (9.10, Unauthorized Remote-Access Tools & P2P/Torrent Clients) was researched and written to close it, grounded in CIS Controls v8.1 and a real CISA/NSA/MS-ISAC advisory (AA23-025A). In the process, a vendor blog's mischaracterization of a *different* CISA advisory was caught and documented (not silently corrected) — consistent with the report's own evidentiary discipline.

5.2 Action-severity mapping (`recommendation_tier.py`)

Maps the statistical (`risk_category`) onto a separately-authored operational specification's R1-R6 action tiers (No action → Escalate to security), reconciling two independently-built classification axes ("how anomalous" vs. "what should a human do").

Review process: rather than accepting the module's validation against its spec's 5 worked examples as sufficient, edge cases the 5 examples never test *together* were constructed and run directly. This surfaced two real bugs:

- An (`Informational`)-tier event with high peer prevalence was incorrectly escalated to R4, contradicting the module's own stated reasoning for why (`Informational`) should never individually escalate.
- A known-bad-category floor (Wazuh CDB list match) was silently overriding R4 redirects — meaning a widespread Shadow-AI tool matching a CDB list would always be bumped to "block," directly contradicting the same project's own research finding that "provide a sanctioned alternative" is the single best-evidenced intervention for that exact category.

Both were fixed, and a 27-test suite (`tests/test_recommendation_tier.py`) was built covering all 5 spec examples, both bug-fix regressions, input validation, and the DataFrame batch-application path — the first automated test coverage in the project.

6. Results Progression

Stage	Metric	Result
Heuristic, event-level	AUC (real scale)	0.6159
Heuristic, day-level (percentile flagging)	F1	0.1067
Trained classifier, event-level	AUC (overall / CERT / CASB)	0.53 / 0.53 / 0.93
Trained classifier, day-level (Random Forest)	F1 (overall / CERT / CASB)	0.9496 / 0.979 / 0.452
GA+DT, pooled fitness	F1 (overall / CERT / CASB)	0.9399 / 0.966 / 0.556
GA+DT, balanced fitness + absolute thresholds	F1 (overall / CERT / CASB)	0.9839 / 0.989 / 0.899

Each row above reflects a specific, documented cause for the change from the row before it — not incremental unexplained drift.

7. Known Limitations and Honest Trade-offs

- **CASB's F1 (0.899) sits at a genuine ceiling for the current model.** Both sources are scored by the same Decision Tree, so they're mathematically constrained to the same finite set of leaf probabilities — a source-specific threshold sweep cannot find anything the combined sweep didn't already test. Further improvement would require a different model architecture for CASB specifically, which was evaluated and explicitly declined (§4.8).
- **The GA's hyperparameter search space is a curated discrete set**, not a continuous range — a deliberate choice to keep every candidate a sane, real option and avoid wasted evaluations on degenerate configurations, at the cost of not exploring the full continuous space.
- `recommendation_tier.py`'s escalation thresholds (`repeat_count`≥3, `peer_prevalence`≥40%) are the specification's own starting estimates, not empirically derived — by the spec's own admission, intended to be tuned once real self-audit outcome data exists.
- **Four of `recommendation_tier.py`'s input signals** (`repeat_count`, `peer_prevalence`, `self_audit_response`, `is_known_bad_category`) **don't exist anywhere upstream yet** — the module is built to use them the moment they do (via `apply_recommendation_tiers()`'s column-detection fallback), but today every row is scored using only `risk_category`.

- **Automated test coverage exists for one module** (`recommendation_tier.py`), 27 tests); the rest of the pipeline has been validated by direct measurement against real data at each stage, but not by a regression-test suite.
-

8. Cross-Role Dependencies (Open)

- **Backend/Infrastructure:** real Wazuh alert samples (for a future `wazuh_loader.py`), the API/database contract for `risk_engine.py`'s output, the real multi-tenant `client_id` scheme.
 - **Detection & Security Engineering:** rule-ID-to-category mapping, a unified MITRE ATT&CK tag vocabulary (the ML pipeline and the Wazuh rule spec currently use two non-overlapping tag sets), the exact schema of a live Wazuh rule hit as a third signal to merge into `risk_engine.py`.
 - **Frontend/Dashboard:** whether per-event drill-down is needed beyond the daily aggregate, and whether recommended fixes should be structured for display rather than a joined string.
-

Document prepared by the ML & Data Engineering role. All results in this document are reproducible from the pipeline code in this repository, run against the data sources described in §2.